Code Generation
Compiler Design (CS 3007)

Sumanta Pyne

Assistant Professor
Computer Science and Engineering Department
National Institute of Technology, Rourkela, INDIA

April 14, 2026

Code Generation using Labeled Tree Method

Source code

F:=A+B-(E-(C+D))

Intermediate code

T1:=A+B
T2:=C+D
T3:=E-T2
T4:=T1-T3
F:=T4

Register Bank, R ={R0,R1,R2,...}
Memory locations, M ={M0,M1,M2,...}
Instruction Set={MOV,ADD,SUB,...}

$$\text{MOV } X,Y; X \leftarrow Y, ((X \in R) \wedge (Y \in R \cup M)) \vee ((X \in R \cup M) \wedge (Y \in R))$$
$$\text{ADD } X,Y; X \leftarrow X + Y, (X \in R) \wedge (Y \in R \cup M)$$
$$\text{SUB } X,Y; X \leftarrow X - Y, (X \in R) \wedge (Y \in R \cup M)$$

Code Generation using Labeled Tree Method

Source code
F:=A+B-(E-(C+D))

Intermediate code
T1:=A+B
T2:=C+D
T3:=E-T2
T4:=T1-T3
F:=T4

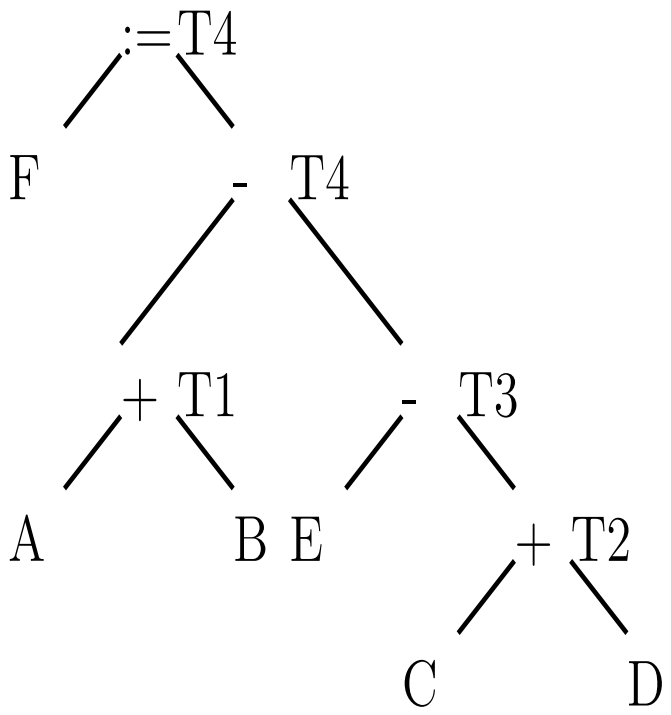
Register Bank={R0,R1,R2,...}
Memory locations={M0,M1,M2,...}

Instruction Set={MOV,ADD,SUB,...}

Tree from Intermediate code

Source code
F:=A+B-(E-(C+D))

Intermediate code
T1:=A+B
T2:=C+D
T3:=E-T2
T4:=T1-T3
F:=T4

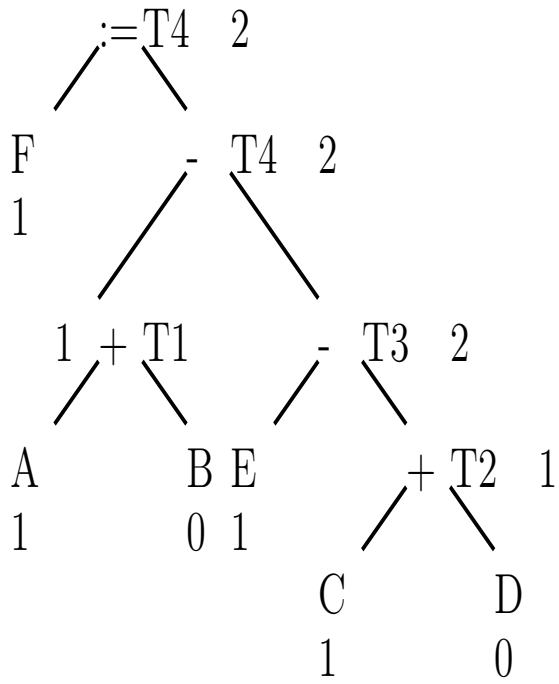


Register Bank={R0,R1,R2,...}
Memory Locations={M0,M1,M2,...}

Labeling Tree

Source code
F:=A+B-(E-(C+D))

Intermediate code
T1:=A+B
T2:=C+D
T3:=E-T2
T4:=T1-T3
F:=T4



Register Bank={R0,R1,R2,...}
Memory Locations={M0,M1,M2,...}
Min. #registers = 2

Code Generation

Source code

F:=A+B-(E-(C+D))

Intermediate code

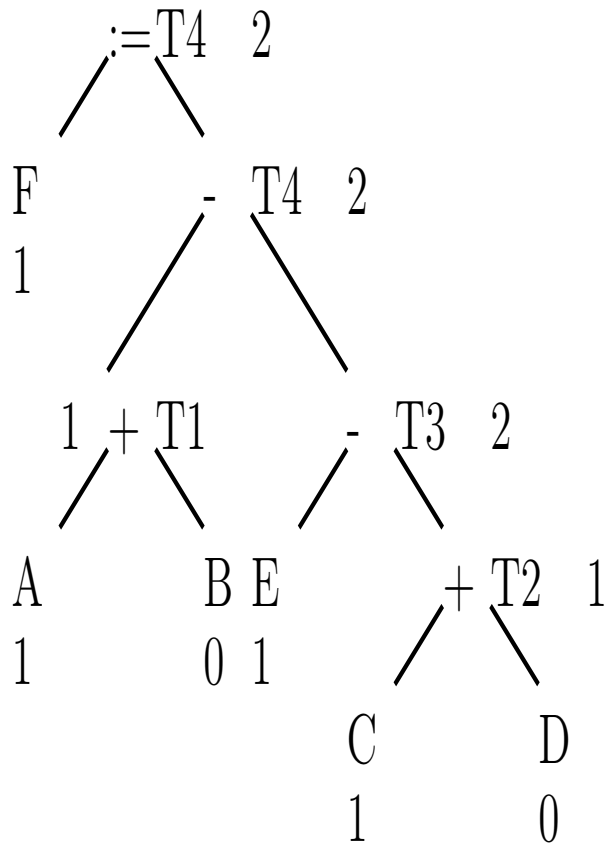
T1:=A+B

T2:=C+D

T3:=E-T2

T4:=T1-T3

F:=T4



Target code

MOV R1,E

MOV R0,C

ADD R0,D

SUB R1,R0

MOV R0,A

ADD R0,B

SUB R0,R1

MOV F,R0

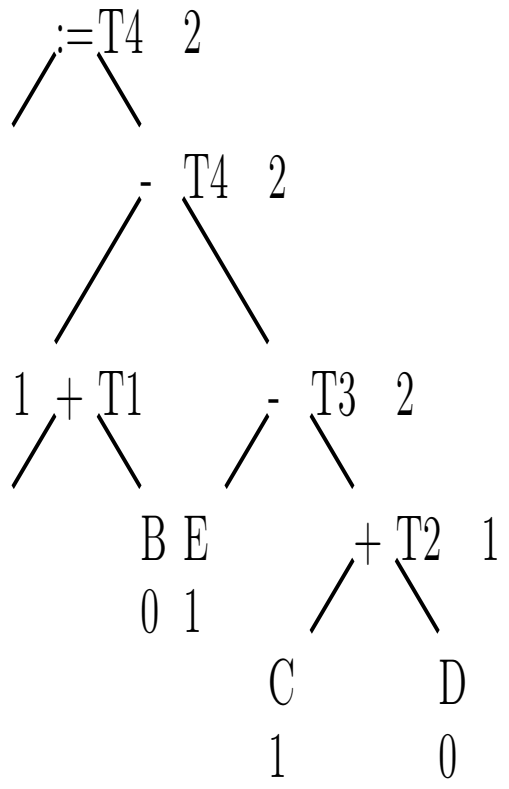
Register Bank={R0,R1,R2,...}

Min. #registers = 2

Number of registers required is less

Source code
F:=A+B-(E-(C+D))

Intermediate code
T1:=A+B
T2:=C+D
T3:=E-T2
T4:=T1-T3
F:=T4



Target code

MOV R0,C; R0←C
ADD R0,D; R0←C+D
MOV M0,R0; M0←C+D
MOV R0,E; R0←E
SUB R0,M0; R0←E-(C+D)
MOV R0,A; R0←A
ADD R0,B; R0←A+B
SUB R0,M0; R0←A+B-(E-(C+D))
MOV F,R0;F←A+B-(E-(C+D))

Register Bank={R0} Min. #registers = 2
Memory locations={M0,M1,...}

Dynamic Programming Code-Generation

- Previous algorithm produces optimal code from an expression tree
- Time taken is a linear function of the size of the tree
- It works for machines in which all computation is done in registers
- Where instructions consist of an operator applied to two registers or to a register and a memory location
- Dynamic programming can be used to extend the class of machines for which optimal code can be generated from expression trees in linear time
- Dynamic programming algorithm applies to register machines with complex instruction sets

Contiguous Evaluation

- The dynamic programming algorithm partitions the problem of generating optimal code for an expression into the subproblems of generating optimal code for the subexpressions of the given expression.
- As a simple example, consider an expression E of the form E1 + E2.
  - An optimal program for E is formed by combining optimal programs for E1 and E2, in one or the other order, followed by code to evaluate the operator +.
  - The subproblems of generating optimal code for E1 and E2 are solved similarly.
- It evaluates an expression E = E1 **op** E2 "contiguously"

Dynamic Programming

- Expression E = E1 **op** E2
- Syntax tree T for E
- T1 and T2 are trees for E1 and E2
- A program P evaluates a tree T contiguously if it
  - First evaluates subtrees of T to be computed into memory
  - Then, it evaluates remainder of T either in the order
    - T1, T2 and then root, or
    - T2, T1 and then root
  - using the previously computed values from memory
- The contiguous evaluation ensures that for any expression tree T
  - there always exists an optimal program
  - that consists of optimal programs for subtrees of the root
  - followed by an instruction to evaluate the root

The Dynamic Programming Algorithm

- Suppose the target machine has r registers
- The dynamic programming algorithm proceeds in three phases
  1. Compute bottom-up for each node n of the expression tree T
     - An array C of costs, in which the ith component C[i] is the optimal cost of computing the subtree S rooted at n into a register, assuming i registers are available for the computation
  2. Traverse T, using the cost vectors to determine which subtrees of T must be computed into memory
  3. Traverse each tree using the cost vectors and associated instructions to generate the final target code. The code for the subtrees computed into memory locations is generated first.
- Each phases run in time linearly proportional to size of expression tree

An example

- Consider a machine having two registers R0 and R1
- Following instructions, each of unit cost:
  LD Ri, Mj // Ri = Mj
  op Ri, Ri, Rj // Ri = Ri op Rj
  op Ri, Ri, Mj // Ri = Ri op Mj
  LD Ri, Rj // Ri = Rj
  ST Mi, Rj // Mi = Rj
- Ri is either R0 or R1, and Mj is a memory location
- Expression *(a-b)+c*(d/e)*

Computing cost vector at leaves

$(a - b) + c * (d/e)$ Max #registers = 2

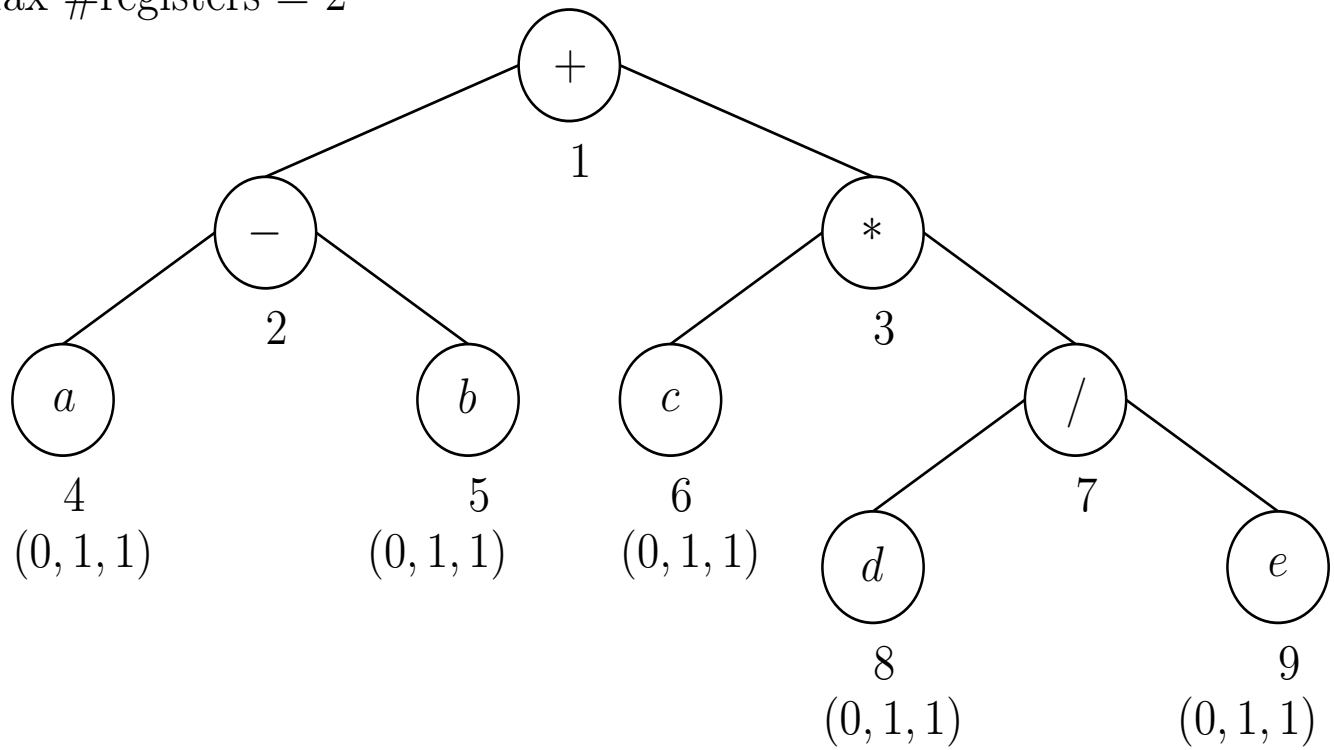
C4[0]=0,C4[1]=1,C4[2]=1

C5[0]=0,C5[1]=1,C5[2]=1

C6[0]=0,C6[1]=1,C6[2]=1

C8[0]=0,C8[1]=1,C8[2]=1

C9[0]=0,C9[1]=1,C9[2]=1



- C[0], the cost of computing 'a' into memory, is 0 – it is already there
- C[1], the cost of computing 'a' into a register, is 1
  - load it into a register with the instruction LD R0, a.
- C[2], the cost of loading a into a register with two registers available is the same as that with one register
- The cost vector at leaf a is therefore (0,1,1)

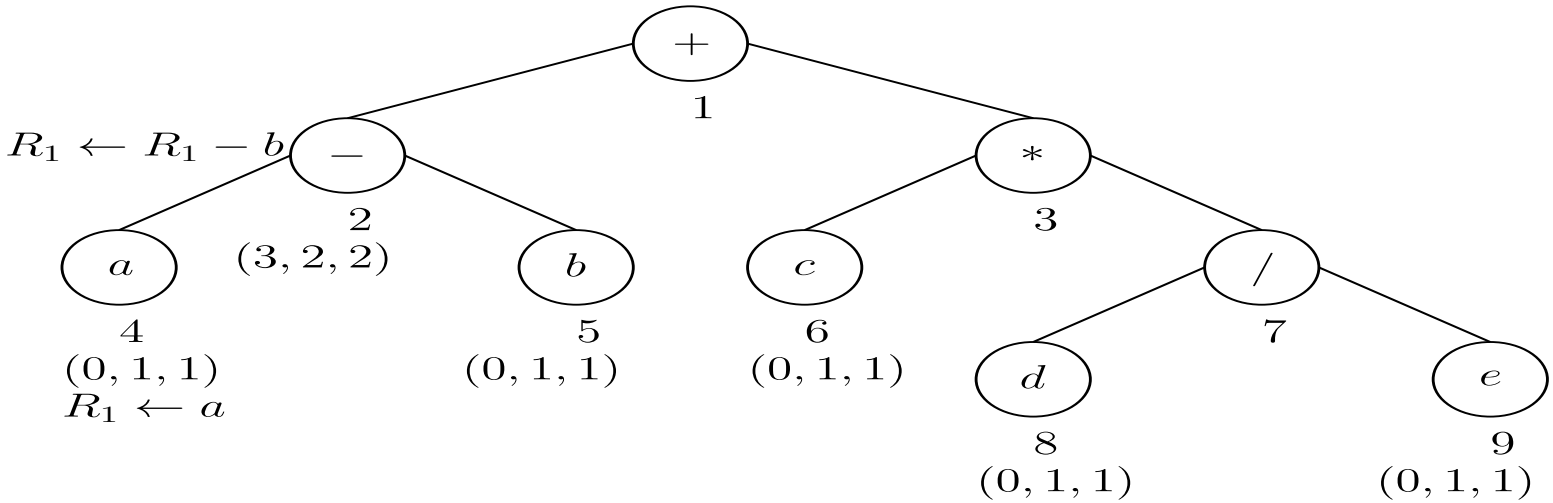
Cost Vector Evaluation

- C[i] = min cost of computing the complete subtree rooted at n with r register available
- Cost of computing a tree into memory = cost of computing tree with all registers + 1 (store cost)
- Traverse and emit code
  - memory computations first
  - rest later, to get optimal cost
- Minimum cost of evaluating root with one register available is
  - minimum cost of computing its right subtree into memory, plus
  - minimum cost of computing its left subtree into the register, plus
  - 1 for the instruction

Cost Vector Evaluation

- Three cases arise depending on order of evaluation
- Case I
  - Compute the left subtree with two registers available into register R0
  - Compute the right subtree with one register available into register R1
  - Use the instruction ADD R0, R0, R1 to compute the root
- Case II
  - Compute the right subtree with two registers available into R1
  - Compute the left subtree with one register available into R0
  - use the instruction ADD R0, R0, R1
- Case III
  - Compute the right subtree into memory location M
  - Compute the left subtree with two registers available into register R0
  - Use the instruction ADD R0, R0, M

Code Generation using Dynamic Programming

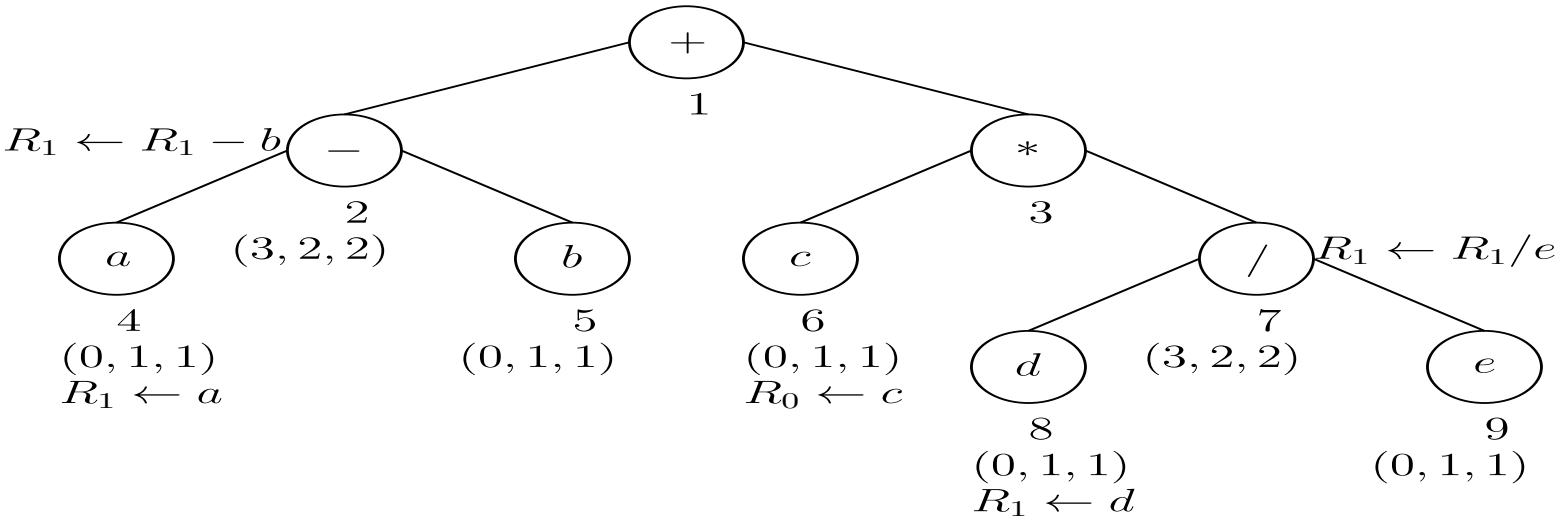


C2[1]=C4[1]+C5[0]+1=2

C2[2]=Min { C4[2]+C5[1]+1,
C4[2]+C5[0]+1,
C4[1]+C5[2]+1,
C4[1]+C5[1]+1,
C4[1]+C5[0]+1}
=Min { 1+1+1,
1+0+1,
1+1+1,
1+1+1,
1+0+1}
=Min {3,2,3,3,2}=2

C2[0]=1+C2[2]=1+2=3

Code Generation using Dynamic Programming

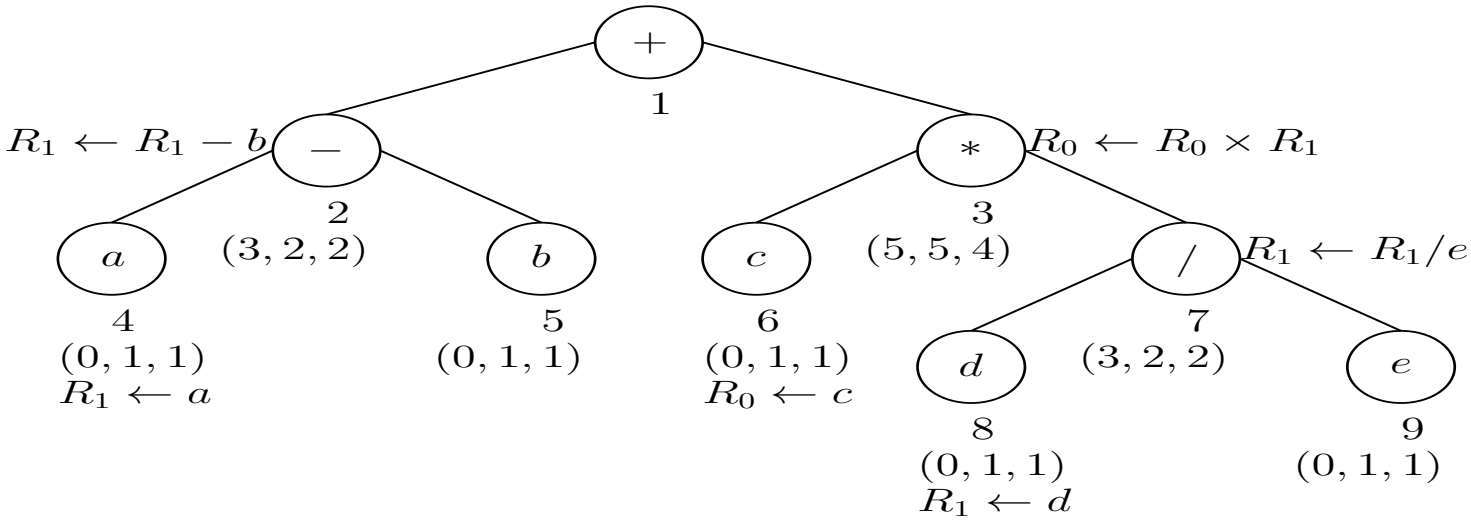


C7[1]=C8[1]+C9[0]+1=2

C7[2]=Min { C8[2]+C9[1]+1,
C8[2]+C9[0]+1,
C8[1]+C9[2]+1,
C8[1]+C9[1]+1,
C8[1]+C9[0]+1}
=Min { 1+1+1,
1+0+1,
1+1+1,
1+1+1,
1+0+1}
=Min {3,2,3,3,2}=2

C7[0]=1+C7[2]=1+2=3

Code Generation using Dynamic Programming

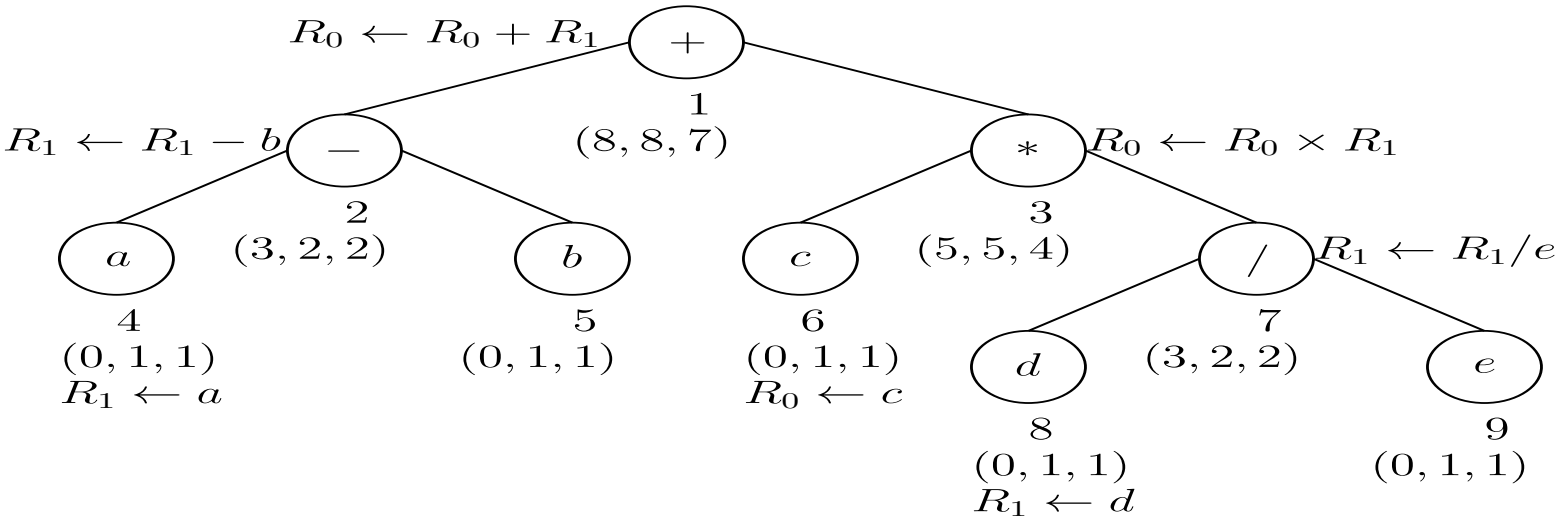


C3[1]=C6[1]+C7[0]+1=1+3+1=5

C3[2]=Min { C6[2]+C7[1]+1,
C6[2]+C7[0]+1,
C6[1]+C7[2]+1,
C6[1]+C7[1]+1,
C6[1]+C7[0]+1}
=Min { 1+2+1,
1+3+1,
1+2+1,
1+2+1,
1+3+1}
=Min {4,5,4,4,5}=4

C3[0]=1+C3[2]=5

Code Generation using Dynamic Programming

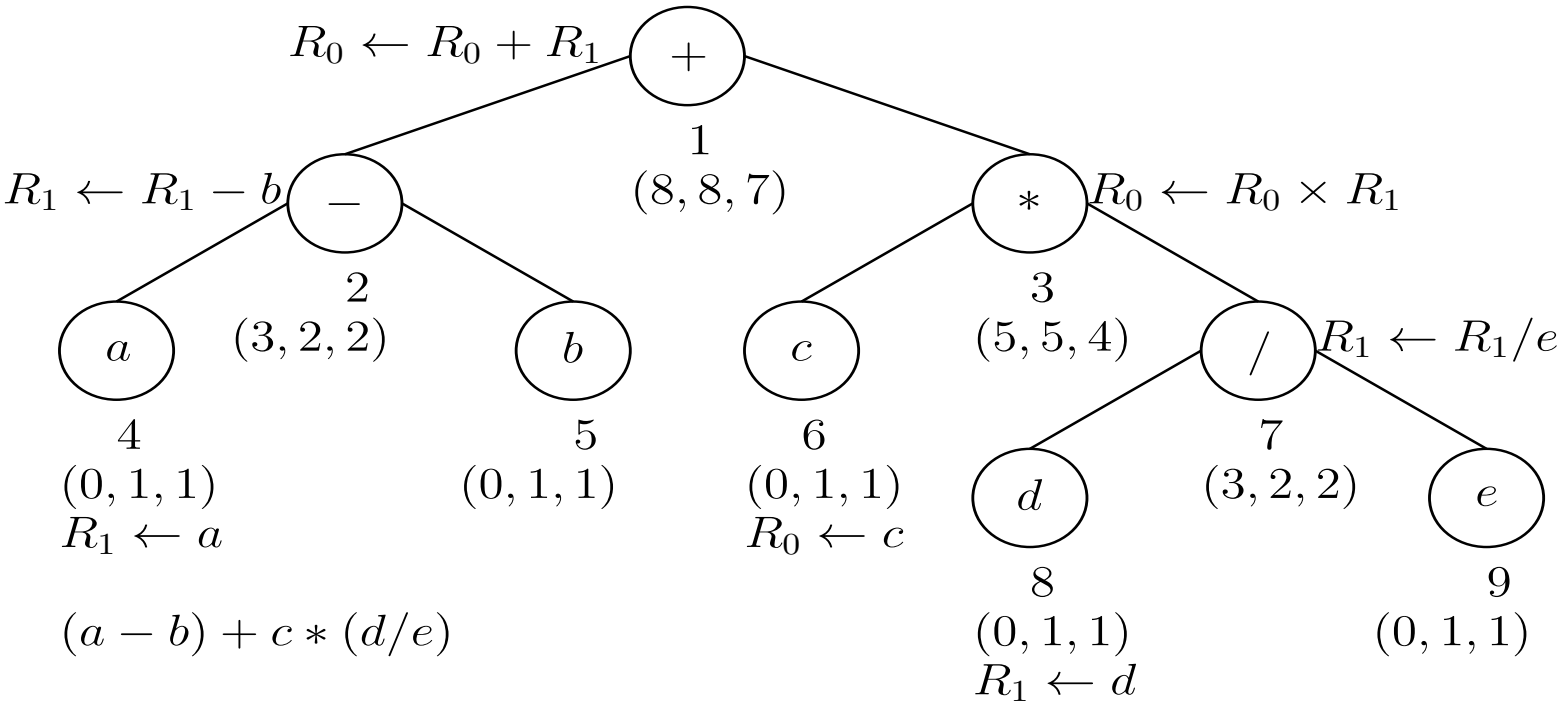


C1[1]=C2[1]+C3[0]+1=8

C1[2]=Min { C2[2]+C3[1]+1,
C2[2]+C3[0]+1,
C2[1]+C3[2]+1,
C2[1]+C3[1]+1,
C2[1]+C3[0]+1}
=Min { 2+5+1,
2+5+1,
2+4+1,
2+5+1,
2+5+1}
=Min {8,8,7,8,8}=7

C1[0]=1+C1[2]=1+7=8

Code Generation using Dynamic Programming



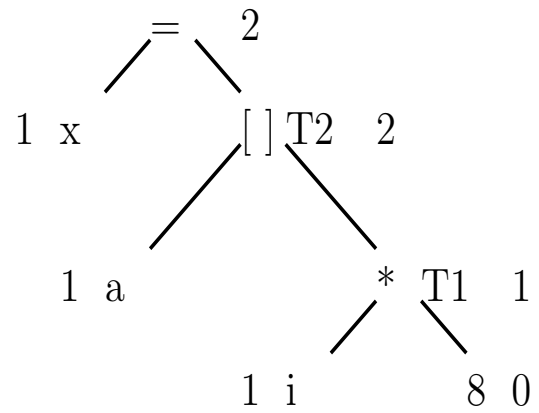
Optimal code for $(a - b) + c * (d/e)$

$LD\ R_0, c; R_0 \leftarrow c$
$LD\ R_1, d; R_1 \leftarrow d$
$DIV\ R_1, R_1, e; R_1 \leftarrow R_1/e$
$MUL\ R_0, R_0, R_1; R_0 \leftarrow R_0 \times R_1$
$LD\ R_1, a; R_1 \leftarrow a$
$SUB\ R_1, R_1, b; R_1 \leftarrow R_1 - b$
$ADD\ R_1, R_1, R_0; R_1 \leftarrow R_1 + R_0$

Target Code Generation for Array Expressions

Source code
x=a[i];

Intermediate code
T1:=i*8
T2:=a[T1]
x:=T2

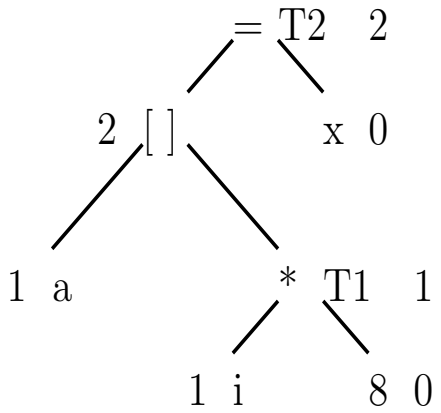


Target code
MOV R1,i; R1←i
MUL R1,8; R1←R1*8
MOV R0,a(R1); R0←a[R1]
MOV x,R0; x←R0

Register Bank={R0,R1,R2}
Memory locations={M0,M1,...}

Source code
a[i]=x;

Intermediate code
T1:=i*8
T2:=x
a[T1]:=T2



Target code
MOV R0,i; R0←i
MUL R0,8; R0←R0*8
MOV R1,x; R1←x
MOV a(R0),R1; a[R0]←R1

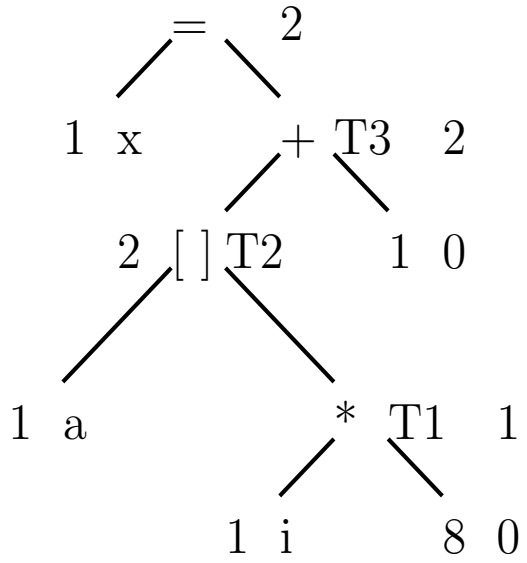
Register Bank={R0,R1,R2}
Memory locations={M0,M1,...}

Target Code Generation for **x=a[i]+1;**

Source code
x=a[i]+1;

Intermediate code
T1:=i*8
T2:=a[T1]
T3:=T2+1
x:=T3

Register Bank={R0,R1,R2}
Memory locations={M0,M1,...}



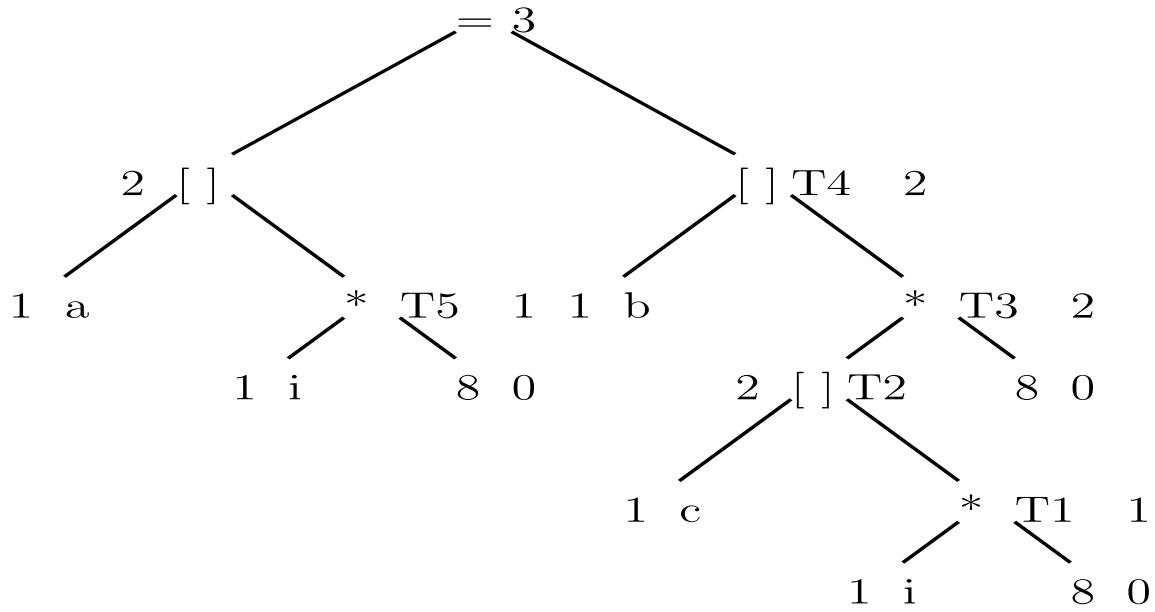
Target code
MOV R1,i; R1←i
MUL R1,8; R1←R1*8
MOV R0,a(R1); R0←a[R1]
ADD R0,1; R0←R0+1
MOV x,R0; x←R0

Target Code Generation for **a[i]=b[c[i]];**

Source code
a[i]=b[c[i]];

Intermediate code
T1:=i*8
T2:=c[T1]
T3:=T2*8
T4:=b[T3]
T5:=i*8
a[T5]:=T4

Register Bank={R0,R1,R2}
Memory locations={M0,M1,. . . }



Target code
MOV R1,i; R1←i
MUL R1,8; R1←R1*8
MOV R0,c(R1); R0←c[R1]
MUL R0,8; R0←R0*8
MOV R1,b(R0); R1←b[R0]
MOV R0,i; R0←i
MUL R0,8; R0←R0*8
MOV a(R0),R1; a[R0]←R1

- Rest are under construction.
  - a[i][j]=b[i][k]*c[k][j];
  - a[i]=a[i]+b[j];
  - a[i]+=b[j];
- You can solve them in similar manner.
- You can also find better solutions and share them with all of us.